Alemeno Frontend Internship Assignment

Custom Marker Detection & Extraction App

Approach Document

Candidate	Intern Applicant
Technology	React Native 0.73 (Android)
Marker Chosen	Marker 1 — 140×140 unit square with top-left anchor
Date	April 2026

1. Project Overview

This document describes the technical approach taken to build an Android React Native application capable of detecting a custom visual marker in real time from a high-resolution camera feed, extracting it with precise orientation correction, and displaying 20 processed 300×300 px captures in a grid UI.

2. Marker Selection & Design

Marker 1 was selected from the provided set. Its design specifications are:

Property	Specification
Overall size	140 × 140 units (square)
Border thickness	20 units on all sides
Border colour	Solid black
Interior	Mostly white (>60% empty)
Orientation anchor	20 × 20 black square, top-left interior corner
Colours used	Black & White only

The orientation anchor is the key distinguishing feature. By placing the small black square only in the top-left interior corner, the marker has a unique signature under all four 90° rotations. This makes orientation correction deterministic and reliable without any machine-learning dependency.

3. Detection Pipeline

The full detection pipeline runs on a background thread using React Native Reanimated worklets, keeping the UI thread responsive at all times.

Step	Operation	Reason
1	RGBA → Greyscale (BT.601)	Reduce data by 75%, faster processing
2	Otsu global threshold	Adaptive to lighting, no manual tuning
3	BFS connected-component blob finder	Find large dark regions (the border)
4	Aspect-ratio filter (±15%)	Marker is square; reject rectangles
5	Area bounds check (1%–80% of frame)	Ignore tiny noise and full-screen fills
6	Interior whitespace check (>60%)	Marker interior must be mostly empty
7	Top-left anchor validation (>55% dark)	Confirm the orientation marker exists
8	Other-corner check (<55% dark)	Prevent false positives from similar shapes
9	Orientation detection	Determine which corner holds the anchor
10	Stability gate (3 consecutive frames)	Reduce blur / partial detection captures
11	Photo capture + crop + rotate + resize	Extract clean 300×300 px output

4. Orientation Correction Strategy

The 20×20 orientation anchor sits just inside the thick border at the top-left corner when the marker is upright (0°). After locating the outer bounding box, the algorithm samples the pixel darkness at each of the four interior corners and picks the darkest one as the anchor location:

Anchor corner	Detected rotation	Correction applied
Top-Left	0° (upright)	No rotation
Top-Right	90° clockwise	Rotate –90° (270°)
Bottom-Right	180°	Rotate 180°
Bottom-Left	270° clockwise	Rotate –270° (90°)

This approach is lighting-invariant because it uses relative darkness (a threshold), not absolute pixel values. It handles all 360° of rotation discretised to 90° steps, which is sufficient for any real-world print orientation.

5. Extraction Accuracy

After the marker is detected and its orientation determined, the photo is captured at the sensor's native high resolution. Extraction follows these steps to guarantee a tightly-cropped, skew-free 300×300 px output:

- Scale bounding box from detection-frame coordinates to full-resolution photo coordinates using the ratio of frame vs photo dimensions.
- Add a 2% safety margin to ensure the full border is included without clipping.

- Crop and simultaneously rotate using react-native-image-resizer (single-pass, zero intermediate re-compression artefacts).
- Resize to exactly 300×300 px using stretch mode — guarantees pixel-perfect output regardless of the original aspect ratio variance.
- Delete all temporary files to keep device storage clean.

6. False Positive Prevention

The multi-condition validation pipeline makes it extremely unlikely to falsely detect non-markers:

Incorrect image type	Rejected by
Plain square outline (no anchor)	Anchor fill < 55% threshold
Marker with anchor in wrong corner	Only top-left zone checked first; orientation step confirms
Rectangle (not square)	Aspect ratio ±15% filter
Small random black blobs	Minimum area (1% of frame) filter
Mostly dark images	Interior whitespace check (>60% white)
Other Marker 2 design	Marker 2 has no top-left anchor blob

7. Performance & Speed

The app is designed to complete the full scan-to-result pipeline well under the 3000 ms target:

Step	Approx. Time
Frame processing (detection)	~30–50 ms per frame (throttled to 150 ms)
Stability gate (3 frames)	~450 ms total wait
Photo capture	~200–400 ms
Crop + rotate + resize	~100–200 ms
Base64 read	~10 ms
Total (typical)	~800–1100 ms per marker

All image processing runs in a Reanimated worklet (JS background thread) — the UI remains smooth at 60 fps during scanning. The stability gate prevents wasted captures but adds a predictable latency that keeps the total time comfortably under 3 seconds.

8. Technology Choices & Rationale

Library	Why chosen
react-native-vision-camera v4	Industry standard for high-res camera in RN; native frame processors run off UI thread; suppo
react-native-reanimated v3	Worklet support moves image processing fully off the JS thread; no dropped UI frames
react-native-image-resizer	Handles crop + rotate + resize in a single native call — faster and no intermediate JPEG re-c
react-native-fs	Lightweight base64 reader; minimal overhead

@react-navigation/stack	Standard navigation; negligible overhead
No ML / TensorFlow	Rule-based detection is faster, fully offline, deterministic, and easier to audit

9. Evaluation Criteria — How Each Is Met

Criterion	How met
Speed (<3000 ms)	Rule-based worklet detection ~50 ms/frame; total capture+extract ~1 s
Orientation robustness	4-corner anchor sampling works for 0°/90°/180°/270°; correction applied during crop
Extraction accuracy	Bounding box scaled to full-res photo; 2% margin; stretch-mode resize to exactly 30
Detection accuracy	Multi-condition pipeline: aspect ratio + area + interior whitespace + anchor presence

10. Deliverables Summary

✓ APK file: android/app/build/outputs/apk/debug/app-debug.apk (built with ./gradlew assembleDebug)

✓ Source code: Full React Native project with README and setup instructions on GitHub

✓ This PDF: explains the approach, design choices, and evaluation criteria mapping

✓ Marker used: Marker 1 (provided) — measurements in Marker1-Measurements.jpg

---

Thank you for reviewing this submission. I look forward to the opportunity to contribute to Alemeno's engineering team.